

Relatório Técnico — API de Agendamento Clínico

Disciplina: Desenvolvimento Seguro de Aplicações Web · Prof. Fabiano Domingues · Engenharia de Computação (Infnet) **Entrega:** Assessment DR2 · AT **Aluno:** José Augusto Nascimento **Vídeo de apresentação (YouTube, não listado):** <https://youtu.be/<COLOQUE-O-LINK-AQUI>>

Como ler este relatório. Cada exercício tem: *decisão*, *justificativa*, *onde está no código* e *evidência* (arquivo em `evidence/` ou teste em `tests/`). Todos os números abaixo vêm de execuções reais, reproduzíveis com `bash scripts/collect_evidence.sh` (e `scripts/run_zap_passive.sh` para o ZAP). Itens da rubrica x onde estão demonstrados: Anexo G. O que ainda depende de uma etapa externa (a execução do workflow no GitHub) está marcado como **PENDENTE** no item 12.4 — nada foi inventado.

Resumo executivo

O que é	API FastAPI para agendamento de consultas: pacientes, profissionais e consultas, com 3 tipos de cliente (frontend JSON, página HTML da recepção, laboratório parceiro M2M) e 3 papéis (admin, recepcionista, profissional). Trata dado de saúde (LGPD art. 11).
Stack	Python 3.13 · FastAPI 0.142 · Pydantic 2 · SQLAlchemy 0.0.47 · PyJWT · bcrypt · Jinja2 · pytest
Antes x depois (Ex. 8/9)	O mesmo conjunto de 10 ataques: 10/10 explorados na versão vulnerável → 0/10 na versão corrigida (<code>evidence/ex08_ex09_antes_depois/</code>)
Testes	197 testes automatizados passando · cobertura 96 % do código da aplicação · 25 ameaças do threat model ligadas a 53 verificações de teste (<code>docs/matriz_ameaca_teste.md</code>)
SAST / SCA	Bandit (≥ MEDIUM): sem achados · pip-audit: 0 vulnerabilidades nas dependências finais — contra 25 vulnerabilidades em 5 pacotes nas dependências fixadas pelo Starter Kit
Auditoria OpenAPI	12 verificações, 0 falhas (<code>docs/ex13_openapi_audit.md</code>)
OWASP ZAP (scan passivo real)	4 scans (baseline / , baseline /web/login , API como recepcionista, API como profissional): 0 alertas bloqueantes . O 1º scan achou 1 achado real (90004, COEP ausente, Baixo) → corrigido → 2º scan sem o achado (item 13.4)
Decisão de deploy (Ex. 13)	GO condicional — 7 condições (item 13.6)

Estrutura da entrega

```
clinica-api-assessment/
├─ main.py                # app, middlewares, registro de routers
├─ routes/                # APIRouter por recurso: users, patients, appointments, availability, oauth, web
├─ models/               # tabelas SQLAlchemy + schemas Pydantic (entrada StrictModel / saída whitelist)
├─ database/             # engine, sessão por DI, consultas parametrizadas (repository.py)
├─ auth/                 # camada de autenticação/autorização separada (jwt, mfa, middleware, rbac, ownership)
├─ core/                 # config (BaseSettings), headers, rate limit, auditoria, CSRF
├─ templates/ static/    # Jinja2 com herança (base → login/agenda)
├─ tests/                # 197 testes (um arquivo por exercício) + matriz ameaça-teste
├─ scripts/              # bootstrap_env, seed, e2e_demo, cvss, audit_openapi, gates, coleta de evidências
├─ .github/workflows/    # security-pipeline.yml (security gate)
├─ docs/                 # threat model, pipeline, CVSS, auditoria OpenAPI, risco residual, roteiro do vídeo
└─ evidence/             # evidências por exercício (prints, payloads, saídas de scan)
```

Exercício 1 — Fundação da API

Decisão. Ambiente virtual isolado (`python3.13 -m venv .venv`), dependências fixadas em `requirements.txt` e aplicação nascida **já modularizada** (`routes/`, `models/`, `database/`, mais `auth/` e `core/`), com um `APIRouter` por recurso e `include_router` registrado em um único lugar (`main.py::ROUTERS`).

Justificativa.

- O enunciado pede evitar o retrabalho de "dezenas de rotas num arquivo só": cada recurso (usuários, pacientes, consultas, disponibilidade, OAuth, web) é um módulo; a política de acesso vive em `auth/` e **não é copiada** entre módulos (`require_roles`, `require_scopes` e `get_*_or_404` são o único ponto de decisão).
- Desvios conscientes do Starter Kit** (que fixava FastAPI 0.77, Pydantic 1.10, `python-jose` e Python 3.10/3.11): 1. versões atuais e corrigidas — o `pip-audit` das dependências do kit acusa **25 vulnerabilidades em 5 pacotes** (ex.: CVE-2024-24762, ReDoS em FastAPI/ `python-multipart`; CVE-2024-33663, confusão de algoritmo no `python-jose`) — `evidence/ex12_pipeline/pip_audit_antes.txt`; 2. `PyJWT` no lugar de `python-jose` (o próprio repositório da disciplina já usa `PyJWT` em `events-api-sql`); 3. rotas síncronas (`def`) em vez de `async def` com `Session` síncrona — o kit bloqueava o *event loop* a cada consulta ao banco, um risco de disponibilidade (A04); 4. `lifespan` no lugar de `@app.on_event`, que é *deprecated*.
- O teste `tests/test_ex01_appointments.py` cobre o caminho de sucesso do recurso `consultas` (criar e listar), conforme pedido, e é o primeiro arquivo de uma suíte que cresce por exercício (`test_ex02...`, `test_ex06...`, ...).

Evidência. `evidence/ex01_fundacao/` (`pytest_ex01.txt`, `pip_freeze_venv.txt`, `swagger_docs.png`).

API de Agendamento Clínico

1.0.0

OAS 3.1

/openapi.json

Pacientes, profissionais de saúde e consultas. Dados de saúde (LGPD art. 11).

Authorize



Users



POST

/user/signin Sign In



POST

/user/mfa/verify Mfa Verify



POST

/user/signup Create User



GET

/user/me Read Me



GET

/user/ List Users



PATCH

/user/{user_id}/deactivate Deactivate User



Patients



POST

/patient/new Create Patient



GET

/patient/ List Patients



GET

/patient/search Search Patients



GET

/patient/{patient_id} Read Patient



Appointments



POST

/appointment/new Create Appointment



GET

/appointment/ Read Appointments



GET

/appointment/agenda Read Agenda



GET

/appointment/{appointment_id} Read Appointment



PUT

/appointment/{appointment_id} Update Appointment



DELETE

/appointment/{appointment_id} Cancel Appointment



Availability (laboratório M2M)



GET

/availability/ Read Availability



OAuth2 (laboratório M2M)



Exercício 2 — Exposição de dados e templates seguros

Decisão 1 — response models com whitelist. A tabela Appointment tem campos de auditoria interna (created_by_user_id, created_at, updated_at, created_from_ip, internal_audit_note). Nenhuma rota devolve a tabela: todas declaram response_model com **lista explícita de campos** (AppointmentPublic, AgendaItem, PatientPublic, UserPublic, AvailabilityResponse). É whitelist, não blacklist: um campo novo na tabela **não vaza** até alguém declará-lo na saída.

- Além disso: CPF sai **maskado** (***.***.***-25); password_hash, mfa_seed e demais campos de segredo nunca aparecem em schema de resposta (verificado também no contrato OpenAPI, check O-06).

- Cada perfil recebe a **menor visão** possível: a recepção recebe `AgendaItem` (sem CPF, telefone, anotações clínicas); o laboratório recebe só horários livres.

Decisão 2 — página HTML com Jinja2 e herança (`templates/base.html` → `login.html`, `agenda.html`), `autoescape` **ligado** por `select_autoescape`, nenhum `|safe`. Em profundidade:

1. **Entrada:** texto livre passa por `whitelist/regex` (sem `<` `>` `&` `{` `}` `\`);
2. **Saída:** `autoescape` do Jinja2 neutraliza mesmo dados já persistidos (legados);
3. **Navegador:** `CSP default-src 'none'; style-src 'self'` — sem nenhum script, nem inline;
4. **Sessão:** cookie `HttpOnly; SameSite=Strict`, formulários com token CSRF.

O risco de não declarar `response_model` (justificativa técnica). Sem `response_model` de saída — ou com `response_model` apontando para a própria tabela, como no Starter Kit, o que filtra exatamente nada — o FastAPI serializa **tudo que a função devolve**: ao retornar o objeto da tabela, vão para o cliente *todas* as colunas — inclusive as que ninguém decidiu expor. Isso causa quatro problemas:

1. **Vazamento imediato:** campos de auditoria (`created_from_ip`, `internal_audit_note`), identificadores internos e, em outras entidades, `password_hash` ou CPF completo passam a fazer parte do contrato público (OWASP API3:2023 *Broken Object Property Level Authorization / Excessive Data Exposure*; CWE-213 e CWE-200). Em dado de saúde, vira dado pessoal exposto sem finalidade (LGPD art. 6º, III — necessidade).
2. **Vazamento futuro e silencioso:** com *blacklist* ou sem filtro, toda coluna **nova** da tabela passa a vaziar no deploy seguinte, sem que ninguém revise — o risco cresce com o tempo. Com *whitelist* o campo novo fica de fora até alguém declará-lo na saída.
3. **Facilita outros ataques:** nomes de campos e IPs internos mapeiam a regra de negócio e a infraestrutura para o atacante, e combinados com BOLA entregam o prontuário inteiro.
4. **Não é detectável em revisão de rota:** o código "funciona"; a falha só aparece olhando a resposta — por isso a auditoria do OpenAPI (check O-06) e os testes de resposta passaram a verificar os campos de saída.

Prova com o **mesmo registro** (`evidence/ex02_templates_xss/sem_vs_com_response_model.txt`), versão sem `response_model` dedicado x com `response_model` dedicado:

```
# Mesmo registro, duas versões. A tabela tem campos internos (created_from_ip, internal_audit_note...) e CPF completo.
```

```
### SEM response_model dedicado (response_model = a própria tabela, como no Starter Kit) – versão vulnerável
```

```
$ GET /appointment/1 -> HTTP 200
```

```
{
  "date_time": "2026-10-05T09:00:00",
  "id": 1,
  "reason": "Retorno de exames",
  "notes": "Paciente refere melhora. PA 120/80.",
  "internal_audit_note": "revisar-depois",
  "patient_id": 1,
  "professional_id": 3,
  "status": "agendada",
  "created_from_ip": "10.0.0.5"
}
```

```
### COM response_model dedicado (AppointmentPublic: whitelist de campos) – versão corrigida
```

```
$ GET /appointment/1 -> HTTP 200
```

```
{
  "id": 1,
  "patient_id": 1,
  "professional_id": 3,
  "date_time": "2026-10-05T09:00:00",
  "status": "agendada",
  "reason": "Retorno de exames",
  "notes": "Paciente refere melhora. PA 120/80."
}
```

```
### SEM response_model dedicado – paciente
```

```
$ GET /patient/1 -> HTTP 200
```

```
{
  "professional_id": 3,
  "cpf": "52998224725",
  "phone": "11999000000",
  "id": 1,
  "name": "Maria da Silva"
}
```

```
### COM response_model dedicado (PatientPublic: CPF mascarado) – paciente
```

```
$ GET /patient/1 -> HTTP 200
```

```
{
  "id": 1,
  "name": "Maria da Silva",
  "cpf": "***.***.***-25",
  "phone": "11999000000",
  "email": null,
  "professional_id": 3
}
```

Onde. `models/appointments.py` (`AppointmentPublic`, `AgendaItem`), `models/patients.py` (máscara), `routes/web.py`, `templates/`, `core/security_headers.py`.

Evidência. `tests/test_ex02_response_models_xss.py` (11 testes), inclusive o que **grava um `<script>` direto no banco** e prova que a página o exibe como texto; e um teste que falha se algum template usar `|safe`.

Dados normais

Clinica · Agendamento

Rita Recepção

Saír

Agenda de 05/10/2026

Data

05/10/2026

Ver

HORÁRIO	PACIENTE	PROFISSIONAL	MOTIVO	STATUS
09:00	Maria da Silva	Dra Helena Costa	Retorno de exames	agendada
10:00	João Pereira	Dra Helena Costa	Consulta de rotina	agendada

Uso interno. Contém dados pessoais protegidos pela LGPD — não compartilhe capturas de tela.

Payload XSS persistido no banco (exibido como texto inerte)

Clinica · Agendamento

Rita Recepção

Saír

Agenda de 05/10/2026

Data

05/10/2026

Ver

HORÁRIO	PACIENTE	PROFISSIONAL	MOTIVO	STATUS
09:00	Maria da Silva	Dra Helena Costa	Retorno de exames	agendada
10:00	João Pereira	Dra Helena Costa	Consulta de rotina	agendada
11:00			Dra Helena Costa	<script>alert("xss armazenado")</script>agendada

Uso interno. Contém dados pessoais protegidos pela LGPD — não compartilhe capturas de tela.

Exercício 3 — Fundamentos de segurança e modelagem inicial

3.1 Triáde CIA aplicada à aplicação

Pilar	Ativo / risco principal	Controles implementados (onde)	Lacuna assumida
Confidencialidade (peso regulatório: LGPD art. 11)	Prontuário, CPF, telefone, agenda; credenciais; segredos	JWT + MFA admin (<code>auth/</code>) · RBAC + ownership dentro da query (<code>auth/ownership.py</code>) · response models com whitelist · CPF mascarado · <code>Cache-Control: no-store</code> · <code>bcrypt</code> · segredos só em <code>.env</code> · logs sem PII/tokens	CPF em texto claro no banco (sem criptografia de coluna) → R-01
Integridade	Prontuário, status de consulta, papéis	<code>extra='forbid'</code> (anti mass assignment) · <code>whitelist/regex</code> · queries parametrizadas · JWT assinado (HS256 fixo, <code>iss/aud/exp/jti</code>) · CSRF · índice único de horário · cancelamento lógico · trilha de auditoria	Auditoria em stdout, não imutável → R-06
Disponibilidade	Agendamento em horário de funcionamento; integração do laboratório	rate limit em 2 níveis · paginação com teto · rotas <code>def</code> (não bloqueiam o event loop) · validação barata antes de <code>bcrypt</code> · <code>deny-by-default</code>	Limitador em memória; tamanho de corpo depende do proxy; SQLite → R-02, R-08, R-09

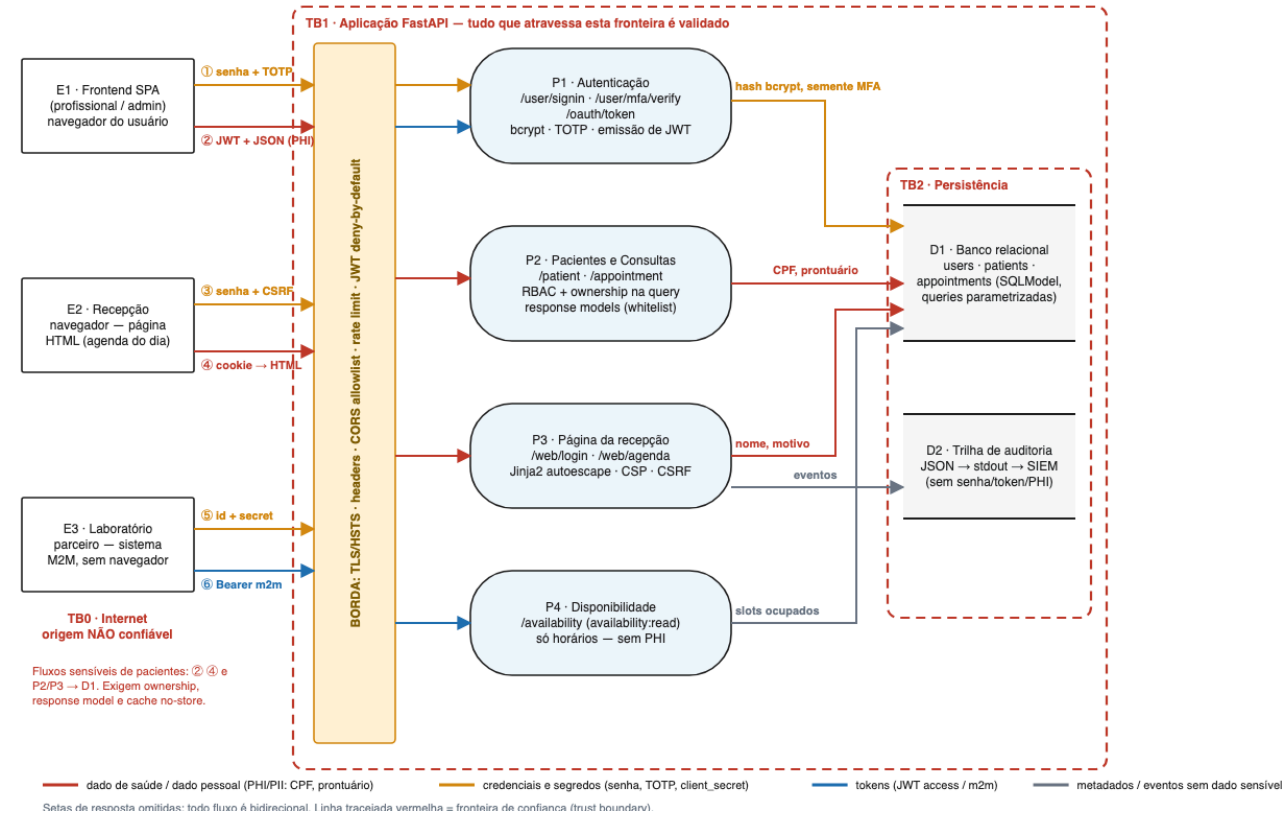
3.2 Frameworks de referência x controles já implementados

Framework	Item	Controle concreto no projeto	Evidência
OWASP Top 10:2021	A01 Broken Access Control	<code>auth/rbac.py</code> , <code>auth/ownership.py</code> , <code>middleware deny-by-default</code>	<code>test_ex06</code> , <code>test_ex09::test_bola_*</code> , <code>test_ex12::test_deny_by_default_*</code>
	A02 Cryptographic Failures	<code>bcrypt</code> , segredo JWT ≥ 32 chars via <code>.env</code> , HSTS, CPF mascarado	<code>test_ex11</code> , <code>test_ex10</code>
	A03 Injection (SQL/XSS)	SQLModel parametrizado, whitelist, Jinja2 autoescape, CSP	<code>test_ex09</code> , <code>test_ex02</code>
	A04 Insecure Design	threat model antes das correções; 404 uniforme; M2M separado de usuário	<code>docs/ex04_threat_model.md</code>
	A05 Security Misconfiguration	CORS allowlist, headers, docs desligadas em produção, <code>--no-server-header</code>	<code>test_ex10</code> , <code>test_ex11::test_producao_exige_endurecimento</code>
	A06 Vulnerable Components	versões atualizadas; <code>pip-audit</code> + Trivy no CI	<code>evidence/ex12_pipeline/pip_audit_*.txt</code>
	A07 Identification & Auth Failures	rate limit, MFA, mensagem genérica, tempo constante	<code>test_ex06</code> , <code>test_ex10</code> , <code>test_ex12</code>
	A09 Logging & Monitoring	<code>core/audit.py</code> (JSON, sem segredos)	<code>test_ex12::test_*.trilha_de_auditoria</code>
	OWASP API Top 10:2023	ownership; JWT/MFA; response models + <code>extra='forbid'</code> ; paginação + rate limit; <code>require_roles</code> ; headers/CORS	idem; auditoria OpenAPI
NIST SSDF (SP 800-218)	PW.1 / PW.2 projetar e revisar com modelo de ameaças	<code>docs/ex04_threat_model.md</code>	—
	PW.4 reutilizar software seguro e mantido	PyJWT em vez de <code>python-jose</code> ; <code>bcrypt</code> direto (sem <code>passlib</code> , cuja última versão, 1.7.4, é de 2020)	<code>requirements.txt</code>
	PW.5 / PW.6 práticas de código seguro e configuração	<code>StrictModel</code> , <code>Settings</code> com validadores, <code>bandit</code>	<code>core/config.py</code>
	PW.7 / PW.8 revisar e testar o código	SAST, 197 testes, auditoria OpenAPI, ZAP	<code>.github/workflows/</code>
	PS.1 / PS.2 proteger o código e as entregas	<i>Actions</i> fixadas por SHA , <code>permissions: contents: read</code> , <code>.env</code> fora do ZIP	<code>test_ex12_ci_gate</code>
MITRE ATT&CK	RV.1 / RV.2 identificar, priorizar e corrigir vulnerabilidades	<code>pip-audit</code> /Trivy, CVSS + impacto de negócio	<code>docs/cvss_priorizacao.md</code>
	T1110 / T1110.004 Brute Force / Credential Stuffing	rate limit 3 baldes, MFA admin	<code>test_ex12::test_credential_stuffing_*</code>
	T1078 Valid Accounts · T1528 Steal Application Access Token	TTL curto, <code>aud / token_use</code> , escopos, desativar conta corta o acesso	<code>test_ex06</code> , <code>test_ex07</code>
	T1190 Exploit Public-Facing Application	validação + parametrização + <code>deny-by-default</code>	<code>test_ex09</code>

Framework	Item	Controle concreto no projeto	Evidência
	T1539 Steal Web Session Cookie	cookie HttpOnly , SameSite=Strict , CSP sem script	test_ex02
	T1552.001 Credentials in Files	segredos só em .env ; Trivy secret ; guarda de teste	test_ex11
MITRE CWE	CWE-639 (IDOR/BOLA) · CWE-915 (mass assignment) · CWE-89 (SQLi) · CWE-79 (XSS) · CWE-798 (credencial fixa) · CWE-352 (CSRF)	os controles acima	evidence/ex08_ex09_antes_depois/

3.3 DFD básico com fronteiras de confiança

DFD nível 1 — API de Agendamento Clínico (Exercício 3)



Trust boundaries. TB0 — Internet/dispositivos (não confiável). TB1 — a aplicação: tudo que a cruza passa pela *camada de borda* (TLS/HSTS, headers, CORS, rate limit, JWT deny-by-default) e por validação de schema. TB2 — persistência: só alcançada por consultas parametrizadas, com credencial vinda do .env .

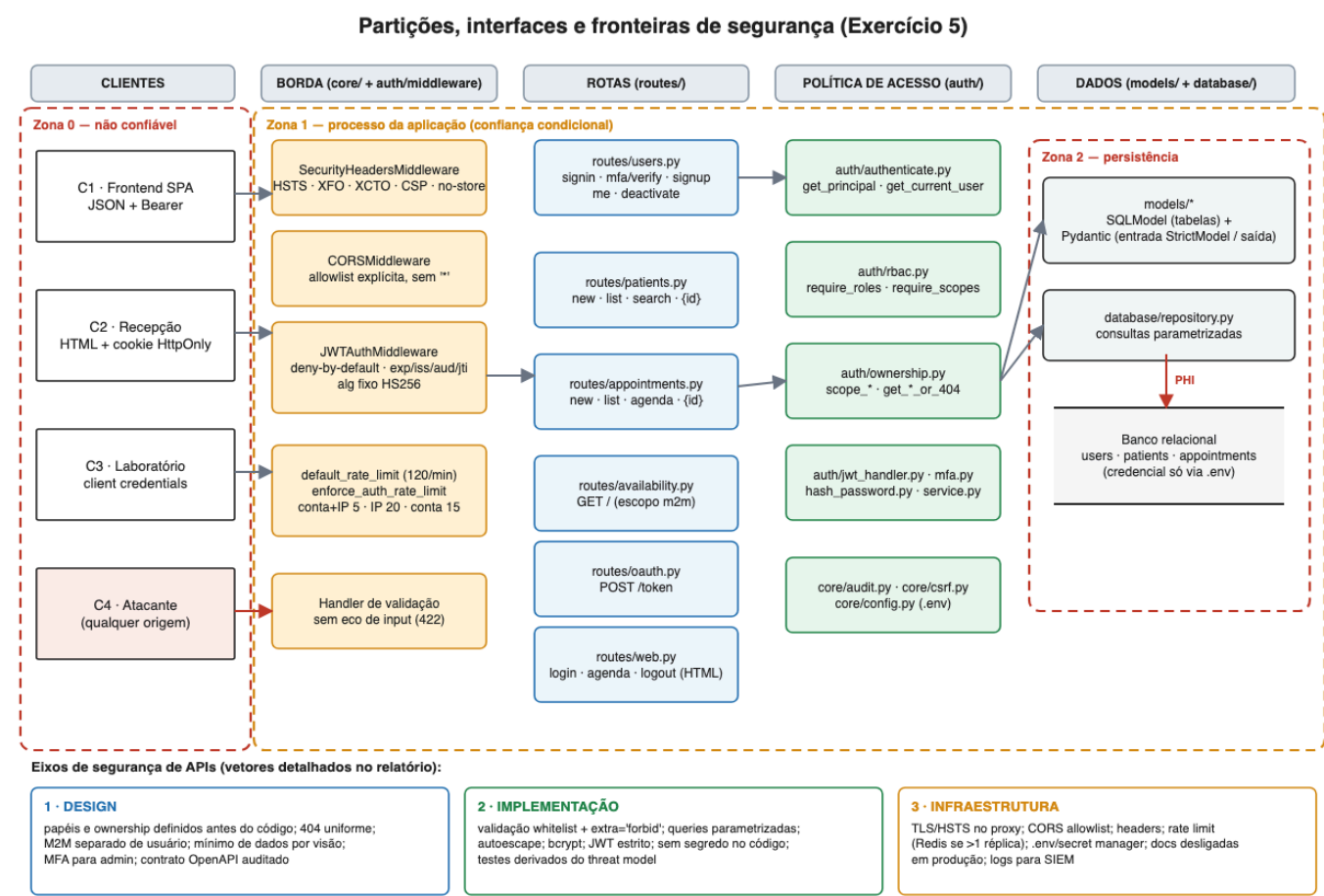
Fluxos de dados sensíveis (pacientes): ② JWT + JSON de pacientes/consultas (saída) e ④ HTML da agenda (saída); P2/P3 → D1 (gravação de CPF, telefone, prontuário, nome e motivo). São os fluxos que exigem *ownership*, *response model*, *no-store* e *escape*. Credenciais (①, ③, ⑤) e hash/semente MFA (P1 → D1) são o segundo grupo crítico.

Exercício 4 — Modelagem de ameaças com STRIDE

O modelo completo — ativos, superfícies de ataque, misuse cases, STRIDE por componente, DREAD e árvore de ataque — está em docs/ex04_threat_model.md . Resumo:

- **9 misuse cases** (MU-01...MU-09), no formato das aulas 8/9 (ator, objetivo, caminho do atacante, resposta do sistema): BOLA de prontuário, cadastro com role=admin , força bruta, XSS armazenado na agenda, SQL injection, abuso do token do laboratório, endpoint esquecido sem proteção, conta admin sem 2º fator, repúdio de cancelamento.
- **STRIDE aplicado a 5 componentes** (o enunciado pede 3): C1 Autenticação · C2 Consultas · C3 Página da recepção · C4 Integração do laboratório · C5 Persistência — **25 ameaças** com ID (T-S1 ... T-X2).
- **DREAD** (escala 0–10 da aula 9) para priorizar: lideram T-T1 mass assignment de role (9,2), T-S1 JWT forjado (8,8) e T-I2 BOLA de leitura (8,4).
- **Rastreabilidade:** cada ameaça aponta para controles e testes em tests/threat_matrix.py ; o teste test_matriz_de_ameacas_aponta_apenas_para_testes_existentes falha se a matriz citar teste inexistente.

Exercício 5 — Arquitetura de segurança e vetores de ataque



Partições (cada uma esconde detalhes atrás de uma interface — aula 10): *Clientes* → *Borda* (core/ + auth/middleware.py) → *Rotas* (routes/) → *Política de acesso* (auth/) → *Dados* (models/ + database/). Três zonas de confiança: **Zona 0** (clientes e atacantes), **Zona 1** (processo da aplicação, confiança condicional) e **Zona 2** (persistência).

Fluxo de uma requisição (padrão do projeto, mesma sequência em todas as rotas):

1. TLS/HSTS no proxy → 2. SecurityHeaders → 3. CORS → 4. JWTAuthMiddleware (token válido? iss/aud/exp/jti, token_use) → 5. rate limit → 6. validação do corpo (StrictModel) → 7. require_roles / require_scopes (**papel** pode chamar a operação?) → 8. get_*, or_404 (**este registro** pertence ao chamador?) → 9. query parametrizada → 10. response_model (só os campos permitidos) → 11. evento de auditoria.

Vetores de ataque nos 3 eixos de segurança de APIs (detalhe e status em docs/ex04_threat_model.md, seção 6):

Eixo	Vetores relevantes	Mitigação	Residual
1. Design	autorização só por papel (BOLA) · token de máquina servindo como token de usuário · operação "esquecida" sem proteção · excesso de dados por visão · enumeração de IDs	papel + dono na query · token_use + escopos · deny-by-default · 1 response_model por visão · 404 uniforme	decisão 404x403 reduz diagnóstico para o cliente legítimo
2. Implementação	injeção (SQL/XSS) · mass assignment · JWT forjado/alg=none · erro que ecoa entrada · CSRF · timing de login	parametrização, whitelist, autoescape · extra='forbid' · JWT estrito · handler de 422 sem input · token CSRF · hash "dummy"	regex de nomes rejeita caracteres raros (usabilidade)
3. Infraestrutura	CORS curinga · ausência de HSTS/XFO/XCTO · força bruta/DoS · segredo no repositório · dependência vulnerável · documentação exposta	allowlist · headers · 3 baldes de rate limit · .env + Trivy/secret · pip-audit · ENABLE_DOCS=false em produção	limitador em memória (1 réplica); TLS e tamanho de corpo no proxy

Exercício 6 — Autenticação e autorização

Decisão. OAuth2PasswordBearer + POST /user/signin (formulário OAuth2); senhas com **bcrypt** (custo 12; rotinas HashPassword.create_hash/verify_hash); **JWT** HS256 com expiração (30 min), iss, aud, nbf, iat, jti obrigatórios e algoritmo **fixo**; **MFA TOTP simulado** obrigatório para admin; autorização **RBAC + ownership**.

MFA (simulado). O algoritmo é o TOTP real (RFC 6238, compatível com Google Authenticator); "simulado" porque não há tela de QR Code nem recuperação. O banco guarda só uma *semente aleatória*; o segredo TOTP é derivado por HMAC com MFA_MASTER_KEY (fora do banco) — um *dump* do banco, sozinho, não gera códigos. Anti-replay: o último passo de 30 s aceito é gravado. Fluxo do admin: `signin` devolve só um `mfa_token` (válido 5 min, inútil como credencial de API); `POST /user/mfa/verify` troca token + código por um `access token`.

RBAC x ABAC x autorização por recurso — por que um híbrido.

Modelo	Responde	Ponto forte	Ponto fraco neste sistema
RBAC puro	"este papel pode chamar a operação?"	simples, auditável, cabe no token	não impede BOLA : dois profissionais têm o mesmo papel e o ID da URL é livre
ABAC puro	"estes atributos permitem?"	flexível	regras espalhadas, difíceis de auditar e testar para só 3 papéis
Por recurso (ownership)	" este registro é seu?"	resolve BOLA	sozinho não restringe as <i>operações</i> por papel
Adotado: RBAC nas rotas + ownership por atributo de dono, aplicado dentro do WHERE	os dois	simples e seguro; política em 2 módulos (<code>auth/rbac.py</code> , <code>auth/ownership.py</code>)	exige disciplina: toda rota por ID usa <code>get_*_or_404</code> (garantido por testes)

Política (conforme o enunciado: apenas profissionais criam e gerenciam consultas dos próprios pacientes):

Operação	admin	recepcionista	profissional
Criar/listar usuários, desativar conta	✓ (com MFA)	✗	✗
Cadastrar paciente	✓	✓ (indica o profissional responsável)	✓ só para si
Ver paciente	✓ todos	✓ todos (CPF mascarado)	✓ só os próprios
Criar/alterar consulta	✗	✗	✓ só para pacientes próprios (<code>professional_id</code> vem do token)
Ler consulta (prontuário) / cancelar	✓ todas	✗	✓ só as próprias
Agenda do dia (sem dados clínicos)	✓	✓	✓ só a própria

O Starter Kit permitia que a recepção criasse consultas e proibia o profissional (`test_medico_nao_pode_criar_consulta`). O enunciado do Ex. 6 diz o contrário ("apenas profissionais ... criem e gerenciem consultas dos próprios pacientes"); segui o **enunciado**. A política está em uma tabela de atalhos (`auth/rbac.py`) e mudá-la é trocar uma linha.

Sessão segura. *Stateless* por JWT, mas `get_current_user` consulta o banco a cada chamada: desativar a conta ou mudar o papel invalida o token **imediatamente**. Cookie da página web: `HttpOnly` , `SameSite=Strict` , `Secure` (obrigatório em produção, por validador de `Settings`).

Teste pedido no enunciado: `tests/test_ex06_authz.py::test_usuario_sem_papel_admin_nao_acessa_rota_restrita_a_admin` — recepcionista e profissional recebem **403** em `GET /user/` e `POST /user/signup`. A mesma suite cobre token expirado, assinatura errada, `alg=none` , outra audiência, papel adulterado, MFA e *replay*.

Evidência. `evidence/ex06_autenticacao_autorizacao/pytest_ex06.txt` (21 testes, inclui MFA pela página web) · `evidence/ex13_capstone/e2e_demo.txt` (passos 2–7).

Exercício 7 — Escopos e integração externa (laboratório)

Fluxo escolhido: OAuth 2.0 *Client Credentials* (RFC 6749 §4.4).

Fluxo	Serve?	Por quê
Authorization Code + PKCE	✗	exige usuário e navegador — o laboratório é um sistema ; PKCE protege cliente público agindo por uma pessoa
Password (ROPC)	✗	o laboratório teria de guardar senha de pessoa; removido no OAuth 2.1
Device Code	✗	para dispositivos sem teclado
Client Credentials	✓	máquina-a-máquina, sem consentimento, sem <i>refresh token</i>

Escopo e claims — o contrato "tecnicamente garantido":

	Usuário (profissional)	Laboratório
<code>token_use</code>	<code>access</code>	<code>m2m</code>
<code>sub</code>	id do usuário	<code>laboratorio-parceiro</code>
<code>papel</code>	<code>role</code> (e conferido no banco)	sem role
<code>escopo</code>	—	<code>availability:read</code>
<code>TTL</code>	30 min	10 min
<code>acessa</code>	rotas de usuário (por papel + ownership)	somente <code>GET /availability/</code>

Se o token do laboratório vazar: só serve para consultar horários livres (sem dado de paciente); expira em 10 min; **403** em qualquer rota de usuário (`token_use` ≠ `access`); pedir escopo fora do contrato devolve `invalid_scope` ; o segredo do cliente só existe como **hash bcrypt** no `.env` ; tentativas são limitadas e auditadas. Em produção o papel do servidor de autorização seria de um IdP (Keycloak, aula 15, com *service account*); o emissor mínimo de `routes/oauth.py` mantém a entrega autocontida com o mesmo contrato (grant, escopos, claims). Endurecimento futuro: mTLS ou `private_key_jwt` (R-07).

Evidência. `evidence/ex07_m2m_escopos/fluxo_client_credentials_curl.txt` (token, claims decodificadas, **403** nas rotas de usuário, `invalid_scope` , `invalid_client`) · `pytest_ex07.txt` (16 testes) · o OpenAPI documenta o esquema `clientCredentials` com o escopo.

Exercício 8 — Identificação de vulnerabilidades (leitura de código)

Método: leitura do código do Starter Kit e da versão pré-hardening (sem scanner) e, para cada hipótese, uma prova de exploração (`evidence/ex08_ex09_antes_depois/attack.py`). A reconstrução da versão pré-hardening (`vulnerable_baseline/app.py`) parte dos defeitos reais do kit e acrescenta os padrões que o enunciado manda procurar.

#	Categoria OWASP	Onde (arquivo:linha)	Padrão vulnerável	Prova
1	A01 / API1 — BOLA (obrigatório)	baseline app.py:170–171 read_appointment (e app.py:147–148 read_patient); kit routes/appointments.py:32 lista todas as consultas	session.get(Appointment, id) sem teste de dono: trocar o ID na URL devolve o prontuário de outro	A2 (leitura), A4 (escrita)
2	A01 / API3 — mass assignment + escalada	kit routes/users.py:12, models/users.py:9	a tabela User é o corpo do cadastro público: {"role":"admin"} vira admin	A1
3	A02 / A07 — segredo fixo e JWT frouxo	kit auth/jwt_handler.py:5,10,23	segredo no repositório (→ forjar token de admin), claim expires própria, except: genérico, sem iss/aud	A6
4	A03 — SQL injection	baseline app.py:142	f"SELECT ... LIKE '%{name}%' (mesmo padrão do github-actions/sast/app.py da aula)	A5
5	A03 — XSS armazenado	baseline app.py:192	{{ a.reason \\ safe }} desliga o autoescape	A7
6	API3 — exposição excessiva	kit routes/appointments.py:14,27, routes/patients.py:15,26	response_model=<tabela> devolve campos internos e CPF completo	A9
7	A07 — força bruta	kit routes/users.py:23	login sem limite de tentativas	A8
8	A05 — misconfiguration	baseline app.py:90	CORS * + credenciais, sem HSTS/XFO/XCTO	A10
9	A06 — componentes vulneráveis	kit requirements.txt	25 vulnerabilidades em 5 pacotes (pip-audit)	pip_audit_antes.txt

Observação que fundamentou o gate do Ex. 12: das 9 linhas, o SAST (Bandit) enxerga **uma** (a SQL injection) e só com `-ll`; o critério do exemplo da aula (`-lll`, só HIGH) **não detectaria nada** (`evidence/ex12_pipeline/bandit_antes.txt`). BOLA, mass assignment e XSS por `|safe` dependem de contexto de negócio — por isso exigem *testes de autorização* e DAST, não só scanner.

Sobre o cenário "paciente autentica e lê o prontuário de outro". O sistema tem três papéis (não há papel "paciente"); modeliei o mesmo defeito entre dois profissionais — a classe de falha (BOLA, CWE-639) e a correção são idênticas.

Exercício 9 — Correção das vulnerabilidades de entrada e saída

Falha (Ex. 8)	Correção centralizada	Onde
BOLA (1)	<code>scope_appointments / scope_patients</code> aplicam o dono dentro do WHERE; <code>get_*_or_404</code> devolve 404 igual para "não existe" e "não é seu" (sem enumeração) e audita a tentativa	<code>auth/ownership.py</code>
Mass assignment (2)	Corpo é StrictModel com <code>extra='forbid'</code> ; papel é Enum; <code>professional_id</code> nunca vem do cliente; cadastro de usuário só por admin	<code>models/base.py</code> , <code>models/*.py</code> , <code>routes/users.py</code>
JWT (3)	Segredo via BaseSettings; HS256 fixo; <code>iss/aud/exp/nbf/iat/jti</code> obrigatórios; decodificação única no middleware JWT centralizado, deny-by-default	<code>auth/jwt_handler.py</code> , <code>auth/middleware.py</code>
SQLi (4)	<code>select().where()</code> parametrizado; termo validado por <code>whitelist (^[A-Za-zÀ-ÿ .'\-]{2,60}\$)</code> ; LIKE com <code>autoescape</code>	<code>database/repository.py</code> , <code>models/base.py</code>
XSS (5)	Whitelist de caracteres na entrada + <code>autoescape</code> do Jinja2 na saída + CSP sem script	<code>models/base.py</code> , <code>templates/</code> , <code>core/security_headers.py</code>
Exposição (6)	<code>response_model</code> <code>whitelist</code> ; CPF mascarado	<code>models/*.py</code>

Endpoint não citado no Ex. 8 com o mesmo padrão (exigido): `GET /patient/{id}` (e `PUT/DELETE /appointment/{id}`). O Ex. 8 cita a leitura de consulta; os demais compartilham o padrão "buscar por ID sem conferir o dono" e passam pelo **mesmo** `get_*_or_404` — um teste percorre todos (`test_ex09::test_bola_em_todos_os_endpoints_por_id`).

Evidência de antes e depois — o *mesmo* ataque (`attack.py`) contra as duas versões:

Exercício 9 — o mesmo ataque, antes e depois da correção

Mesmo conjunto de 10 requisições (attack.py) contra a versão vulnerável (:8001) e a corrigida (:8000). **Antes: 10/10 explorados · Depois: 0/10 explorados.**

ID	Categoria OWASP / ataque	ANTES	DEPOIS	Evidência (versão corrigida)
A1	A01 / API3 — mass assignment + escalada de privilégio POST /user/signup sem login, com role=admin	EXPLORADO	BLOQUEADO	HTTP 401 ("detail":"Credenciais ausentes ou inválidas.")
A2	A01 / API1 — BOLA (leitura) GET /appointment/1 por outro profissional	EXPLORADO	BLOQUEADO	HTTP 404 ("detail":"Consulta não encontrada.")
A3	A01 / API1 — BOLA (mesmo padrão, outro endpoint) GET /patient/1 por outro profissional	EXPLORADO	BLOQUEADO	HTTP 404 ("detail":"Paciente não encontrado.")
A4	A01 / API1 — BOLA (escrita) PUT /appointment/1 por outro profissional	EXPLORADO	BLOQUEADO	HTTP 404 ("detail":"Consulta não encontrada.")
A5	A03 — SQL injection GET /patient/search?name=' OR '1'='1	EXPLORADO	BLOQUEADO	HTTP 422 ("detail":{"loc":["query"],"name"},"msg":"String should match pattern '{A-Za-zÀ-ÖØ-öø-ý .\\-} (2,60)\$'", "type... l linhas devolvidas: 0 (Paulo
A6	A02 / A07 — segredo JWT fixo => token forjado GET /user/ com JWT de admin assinado com o segredo do repositório	EXPLORADO	BLOQUEADO	HTTP 401 ("detail":"Credenciais ausentes ou inválidas.")
A7	A03 — XSS armazenado motivo da consulta com <script> exibido na página da recepção	EXPLORADO	BLOQUEADO	gravação: HTTP 422 {"detail":{"loc":["body","reason"],"msg":"String should match pattern... l <script> cru na página: False
A8	A07 — força bruta sem limite	EXPLORADO	BLOQUEADO	status das 30 tentativas: 401 x5, 429 x25

Saídas brutas: resultado_antes.txt (10/10 explorados) e resultado_depois.txt (0/10). Um achado honesto: o payload x' UNION SELECT cpf FROM patients — passa pela whitelist (só tem letras, espaço, apóstrofo e hífen) — quem o neutraliza é a parametrização, e há teste para cada camada (test_busca_com_payload_sql_e_barrada_pela_whitelist e test_payload_que_passa_na_whitelist_e_neutralizado_pela_parametrizacao). Defesa em profundidade, não uma regra só.

Por que um middleware + dependências, e não só uma coisa. O middleware resolve *quem* é (e dá o deny-by-default para endpoints novos); as dependências resolvem *o que pode* (papel, escopo, dono). O teste test_deny_by_default_em_todas_as_rotas percorre todas as rotas registradas sem token; abrir uma rota exige editar a allowlist PUBLIC_PATHS, que fica visível no code review (e é fixada por outro teste).

Exercício 10 — Hardening de rede e proteção contra abuso

- CORS com allowlist explícita** (CORS_ALLOWED_ORIGINS, lista JSON no .env). Settings **recusa** "*" e origens sem esquema; allow_credentials=False (o frontend usa Bearer, não cookie cross-site); métodos e cabeçalhos também listados.
- Cabeçalhos de segurança** em toda resposta, inclusive 401/422/429 (o middleware de headers é o mais externo): **HSTS** (max-age=63072000; includeSubDomains), **X-Frame-Options: DENY**, **X-Content-Type-Options: nosniff**, mais CSP restritiva, Referrer-Policy, Permissions-Policy, **COOP/COEP/CORP** e Cache-Control: no-store (o COEP entrou depois do primeiro scan do ZAP — item 13.4). A CSP é mais frouxa só para /docs (precisa do CDN do Swagger) — e a documentação é **desligada em produção** por validador.
- Rate limit diferenciado.** Padrão: 120 req/min por IP. **Login / MFA / token M2M**, em três baldes independentes:

Balde	Limite	Para quê
conta + IP	5/min	força bruta a partir de uma origem
IP	20/min	credential stuffing (muitas contas, um IP) sem punir uma recepção atrás do mesmo NAT
conta (qualquer IP)	15/min	teto para força bruta distribuída

Ajustes feitos por falhas que o desenvolvimento revelou: (1) um limite só por IP bloqueava a recepção inteira; (2) um limite só por conta deixaria **um atacante travar o login de qualquer usuário** (DoS por bloqueio). A chave composta *conta+IP* resolve os dois, e há teste para cada caso (test_atacante_nao_consegue_travar_o_login_do_usuario_legitimo_em_outro_ip).

Limitações. HSTS só tem efeito sobre HTTPS (TLS termina no proxy); o contador é em memória do processo (R-02).

Evidência. evidence/ex10_hardening/ — headers_cors_curl.txt, rate_limit_login_curl.txt (5 × 401, depois 429 com Retry-After) e pytest_ex10.txt (17 testes).

Exercício 11 — Persistência segura

- SQLModel** (tabelas) com **toda consulta parametrizada** (select().where()); nenhuma concatenação — protegido por um teste que varre o código por padrões de SQL montado por texto e por um teste que compila uma consulta com payload e mostra o valor como **parâmetro ligado**.
- Sessão por injeção de dependência** (Depends(get_session)); um teste troca a dependência por outro banco e prova que as rotas a usam.
- Credenciais fora do código.** core/config.py (BaseSettings) lê .env; os campos sensíveis **não têm valor padrão** — sem .env a aplicação não sobe (fail closed). Validadores: chave JWT ≥ 32 caracteres, só HS256, CORS sem curinga, e em produção bcrypt ≥ 12, Secure ligado e /docs desligado. SecretStr evita vaziar segredo em repr/logs. A entrega leva só .env.example (placeholders), e o .env real está no .gitignore.

- **Integridade no banco.** `UNIQUE` em e-mail e CPF; **índice único parcial** impede dois horários ativos do mesmo profissional (cancelada libera o horário); `IntegrityError` vira 409.
- O Starter Kit já usava `SQLModel/SQLite`; a "migração" aqui é tornar a camada **configurável e segura**: trocar de `SQLite` para `PostgreSQL` é mudar `DATABASE_URL` (e adicionar migrações Alembic — ver R-09).

Evidência. `tests/test_ex11_persistence.py` (11 testes) · `.env.example`.

Exercício 12 — Pipeline DevSecOps e auditoria automatizada

Documento completo: `docs/ex12_pipeline_devsecops.md`. Arquivo do pipeline: `.github/workflows/security-pipeline.yml` (cópia em `evidence/ex12_pipeline/`).

12.1 Em que fase do SDLC cada tipo de ferramenta atua — e por quê

Tipo	Ferramenta	Fase	Bloqueia?	Justificativa (ligada ao histórico de vulnerabilidades)
Segredos	Trivy <code>secret</code>	commit/PR	sim (qualquer)	F-02: segredo JWT fixo no repositório foi P0; descoberto tarde, o segredo já está no histórico do git
SAST (estática)	Bandit <code>-ll</code>	PR (segundos)	MEDIUM ou maior	F-03: SQL injection por f-string. Barato, determinístico, roda sem aplicação. <code>-lll</code> não a pegaria (prova no item 12.2)
SCA (dependências)	Trivy <code>vuln</code> + <code>pip-audit</code>	PR e semanal	HIGH/CRITICAL e qualquer CVE com correção	F-08: o kit trazia 25 CVEs. CVE nasce sem commit → o agendamento semanal é indispensável
Testes de segurança (pytest)	suíte derivada do threat model	PR	qualquer falha	F-01/04/05/06 (BOLA, mass assignment) não são vistos por scanner : dependem de regra de negócio
DAST passivo	OWASP ZAP baseline + API (<code>-S</code>)	após subir o app (PR/main)	regras <code>FAIL</code> ou risco ≥ Médio	F-09/F-11: headers, CORS, cookies, CSRF só existem com a aplicação rodando
DAST ativo/autenticado	ZAP em staging	pré-release (manual/semanal)	revisão humana	ataca de verdade (fuzz, injeção): nunca em produção, nunca com dado real de paciente
IAST (interativa)	agente instrumentado (ex.: Contrast, Datadog IAST)	QA/staging, durante a suíte de integração	—	enxerga fluxo de dado até a query dentro da aplicação; precisa de agente e carga de testes. Lacuna assumida : sem agente neste projeto — compensada pelos testes de autorização + ZAP autenticado

12.2 Critério do security gate (decidido e justificado)

Bloqueia o merge quando houver: (1) **qualquer teste de segurança falhando** (tolerância zero — são os que cobrem a classe de falha que scanners não veem); (2) **segredo detectado**; (3) **SAST ≥ MEDIUM**; (4) **SCA: HIGH/CRITICAL (Trivy) ou qualquer CVE com correção disponível (pip-audit)**; (5) **DAST: regra marcada FAIL em `.zap/rules.tsv` ou risco ≥ Médio**. Achados abaixo disso geram relatório, não bloqueio.

Em termos de CVSS: **score ≥ 7,0 sempre bloqueia**; **4,0–6,9 bloqueia quando afeta confidencialidade de dado de saúde** (impacto de negócio 4) — o que é garantido por testes, já que nenhuma ferramenta calcula impacto de negócio sozinha; **< 4,0 é registrado**.

Por que esse limiar, e não o do exemplo da aula (HIGH apenas): rodei o Bandit nas duas versões. Com `-lll` a versão vulnerável **passa** (exit 0); com `-ll` é **bloqueada** pela SQL injection (B608 é *Medium*). Em dado de saúde, uma injeção de SQL é catastrófica mesmo sendo "Medium" para o scanner — por isso o corte em MEDIUM (`evidence/ex12_pipeline/bandit_antes.txt`). O gate também **se testa**: o job `gate-regression` falha se o Bandit deixar a versão vulnerável passar.

12.3 Priorização: CVSS + impacto de negócio

Tabela completa (11 vulnerabilidades, vetor CVSS 3.1, score calculado por `scripts/cvss.py` — validado contra scores de referência por testes —, impacto de negócio 1–4 e prioridade P0–P3) em `docs/cvss_priorizacao.md`, incluída no Anexo A. Destaque: a **BOLA de leitura tem CVSS 6,5 ("Média") mas é P1** — vazar prontuário de terceiro é fato gerador de notificação à ANPD (LGPD art. 48); o CVSS sozinho a subestimaria.

12.4 Security gate em GitHub Actions

`security-pipeline.yml`: jobs `sast`, `sca-and-secrets`, `tests`, `gate-regression`, `dast-passive` e `security-gate` (o único check obrigatório no *branch protection*; só libera se **todos** terminarem em `success` — `skipped` bloqueia, lógica testada em `scripts/ci_gate.py`). Boas práticas aplicadas: *Actions* fixadas por **SHA de commit** (o exemplo da aula usava `trivy-action@master`, mutável — risco de cadeia de suprimentos; há teste que reprova qualquer *action* não fixada), `permissions`: `contents: read`, `concurrency`, execução semanal, segredos de teste gerados no próprio *runner*.

Honestidade sobre a execução. Não tenho como executar o workflow no GitHub a partir deste ambiente. Validei a sintaxe (YAML), os SHAs (resolvidos pela API do GitHub) e **executei localmente os mesmos comandos** (`bash scripts/security_gate_local.sh` → `SECURITY GATE: LIBERADO`; saída em `evidence/ex12_pipeline/gate_local_saida.txt`; o ZAP real, em `evidence/ex13_capstone/zap/`). **PENDENTE**: a execução no Actions, o *print* do check `SECURITY GATE` verde e ligar o *branch protection* exigindo esse check — é ele que efetivamente impede o merge (README, "Checklist de entrega"). A sintaxe do cabeçalho `Authorization` para o ZAP dentro da *action* (`cmd_options`) não foi exercitada; localmente, com `docker run`, funcionou.

12.5 Testes de autorização expandidos a partir do threat model

O teste do Ex. 6 virou uma suíte organizada pelas ameaças do Ex. 4: `tests/test_ex12_threat_vectors.py` e a matriz `tests/threat_matrix.py` (25 ameaças → 53 verificações, todas verdes — `docs/matriz_ameaca_teste.md`). Cobrem, entre outros: `deny-by-default` em **todas** as rotas, JWT forjado/expirado/ `alg=none`/outra audiência, BOLA em leitura/escrita/cancelamento, mass assignment, escalada de papel, token M2M × token de usuário, replay de MFA, trilha de auditoria, *credential stuffing* e anti-lockout.

Evidência. `evidence/ex12_pipeline/` (`bandit_antes/depois`, `pip_audit_antes/depois`, `gate_local_saida`, `security-pipeline.yml`, `pytest_ex12.txt`).

Exercício 13 — Capstone: auditoria final e rastreabilidade

13.1 Aplicação final

Autenticação (bcrypt, JWT, MFA, deny-by-default), validação (`extra='forbid'`, whitelist), headers e CORS, persistência segura (SQLModel parametrizado, `.env`), M2M com escopo, página HTML segura e trilha de auditoria — tudo na mesma organização modular. Demonstração ponta a ponta: `python scripts/e2e_demo.py` (saída em `evidence/ex13_capstone/e2e_demo.txt`).

13.2 Testes automatizados com mocking (entrada e autorização)

`tests/test_ex13_unit_mock.py` — 46 testes **unitários** que substituem as dependências com `unittest.mock`:

- **O teste inicial do Ex. 1, agora com mocks:** `TestEx01ComMocks` reexecuta o caminho de sucesso (criar e listar consulta) com **sessão simulada** (`MagicMock`), conferindo `add/commit/refresh`, o `professional_id` vindo do usuário e que a SQL gerada na listagem já filtra pelo dono e tem `LIMIT`;
- **Entrada:** schemas Pydantic isolados (extra proibido, regex, CPF, senha > 72 bytes, data passada/fora de bloco/com fuso, código MFA);
- **Autorização:** `require_roles` e `require_scopes` chamados diretamente com *users* simulados e *audit* espiado; `get_appointment_or_404` com **sessão simulada** (`MagicMock`) provando 404 + alarme de BOLA; a *rota* `create_appointment` chamada diretamente com sessão que lança `IntegrityError` (→ 409) e conferindo que o `professional_id` salvo é o do token;
- **Criptografia/tempo com relógio e bcrypt simulados:** expiração de JWT com `datetime` congelado, `verify_hash` tratando erro do bcrypt, usuário inexistente/inativo gastando o mesmo custo de hash, TOTP com janela e *replay*, *rate limiter* com relógio controlado.

Resultado da suíte completa: **197 passed**, cobertura **96 %** (`evidence/ex13_capstone/pytest_suite_completa_com_cobertura.txt`).

13.3 Auditoria da especificação OpenAPI

`python scripts/audit_openapi.py` (também roda como teste) — resultado em `docs/ex13_openapi_audit.md` e no Anexo C: **12 verificações, 0 falhas**. A auditoria **achou três lacunas reais que foram corrigidas**: `cpf` sem limite declarado; `user_id` sem mínimo; e o formulário de login do FastAPI (`OAuth2PasswordRequestForm`) sem `maxLength` — trocado por um formulário próprio, compatível com o botão *Authorize*. Também provei que os checks detectam o problema (testes que injetam um endpoint público esquecido, um corpo aberto e um campo interno em resposta).

13.4 Scan passivo do OWASP ZAP — executado e interpretado

Como foi executado (`bash scripts/run_zap_passive.sh`; saídas reais em `evidence/ex13_capstone/zap/`): OWASP ZAP 2.17.0 (imagem oficial `ghcr.io/zaproxy/zaproxy:stable`, digest `sha256:781a2bda...5081ef`) em Docker/Colima, contra a API local em `http://host.docker.internal:8000`, com `COOKIE_SECURE=true` (como em produção) e `/docs` ligado apenas para o ZAP importar o OpenAPI. **Quatro scans, todos passivos**:

Scan	Como	URLs	Regras que rodaram sem alerta (PASS)	Alertas
<code>baseline</code>	spider + regras passivas a partir de <code>/</code>	3 (a raiz devolve JSON sem links)	66	1
<code>baseline_web</code>	idem, a partir de <code>/web/login</code> (página HTML, formulário, cookie)	8	64	4
<code>api_recepcao</code>	<code>zap-api-scan -S</code> : importa o <code>openapi.json</code> e visita as rotas autenticado como recepcionista	27	118	3
<code>api_helena</code>	idem, autenticado como profissional	27	118	3

Prova de que foi passivo: a API recebeu **60 requisições** durante os scans e **nenhuma** com payload de ataque (SQLi, XSS, traversal, *time-based*) — prova_scan_passivo_log_de_acesso.txt. A autenticação do ZAP funcionou: as rotas protegidas responderam **403 de RBAC** (não 401) e `GET /user/me` respondeu 200.

Resultado: 0 High, 0 Medium; na 1ª execução, 1 Low — que foi corrigido; as demais ocorrências são informativas. O *gate* (`zap_report.py gate`) libera.

O achado real e sua correção (antes → depois):

Achado (1ª execução)	90004 <i>Cross-Origin-Embedder-Policy Header Missing or Invalid</i> — risco Baixo , CWE-693, em <code>GET /web/login</code> (<code>execucao_1_antes_da_correcao/</code>)
OWASP	A05:2021 <i>Security Misconfiguration</i>
Causa	a aplicação já enviava COOP e CORP, mas não COEP; eu havia mapeado a regra 90004 como cobrindo só COOP/CORP — o scan real mostrou que o mapeamento estava incompleto
Correção	<code>Cross-Origin-Embedder-Policy</code> : <code>require-corp</code> em todas as respostas, exceto <code>/docs</code> (o Swagger carrega scripts de CDN) — <code>core/security_headers.py</code> ; página confirmada renderizando com COEP
Teste	<code>test_ex10_hardening::test_coep_presente_nas_respostas_e_ausente_na_documentacao</code>
Re-scan	2ª execução: PASS: ... [90004] em <code>log_baseline_web.txt</code> — achado resolvido

Interpretação dos alertas que permanecem (todos *Informational*, aceitos com justificativa) — tabela completa e gerada a partir do JSON em `docs/ex13_zap_correlacao.md` (Anexo D):

Alerta	O que significa aqui	Decisão
10049 <i>Non-Storable Content</i> (17x)	é o efeito do <code>Cache-Control</code> : <code>no-store</code> imposto de propósito (T-I3: dado de saúde não pode ficar em cache de proxy/navegador)	aceito — confirma o controle
10049 <i>Storable and Cacheable Content</i> (<code>/static/app.css</code>)	só o CSS público é cacheável, sem dado de paciente	aceito

Alerta	O que significa aqui	Decisão
100000 Client Error response (30x)	403 = RBAC negando o perfil (recepção em /appointment/*, ambos em /user/*, /availability sem escopo m2m); 422 = validação/ extra='forbid'; 404 = ownership/inexistente; 400 = invalid_scope — é a autorização funcionando , vista de fora	aceito
10111 Authentication Request Identified	registra que POST /user/signin e /web/login existem	aceito
10112 Session Management Response Identified	identifica o cookie csrf_token (HttpOnly, SameSite=Strict, Secure) como token de sessão — é o controle CSRF	aceito

O que o silêncio do ZAP comprova. 15 regras rodaram e não alertaram (PASS no log): X-Frame-Options/anti-clickjacking (10020), X-Content-Type-Options (10021), HSTS (10035), CSP (10038), CORS (10098), cookie HttpOnly (10010), Secure (10011) e SameSite (10054), anti-CSRF (10202), vazamento de versão por Server (10036) e X-Powered-By (10037), erro detalhado (90022), cache-control (10015), Permissions-Policy (10063) e isolamento de site (90004, após a correção). Só vale como prova porque o conversor exige o PASS no log — alerta ausente porque a regra nem rodou não prova nada.

Limitações do que o ZAP viu (para não superestimar o scan):

- Passivo não prova ausência de injeção nem de BOLA. Ele só observa respostas; autorização é lógica de negócio. Essas classes estão provadas por testes e pelos ataques A1–A10 (item 9) — é exatamente por isso que o pipeline tem as duas camadas.
- Cobertura de conteúdo: o ZAP monta as requisições com valores de exemplo do OpenAPI (id=10, filtros vazios) e quase nunca satisfaz a validação — vê 403/404/422, não respostas com dado de paciente. Como profissional, as rotas de consultas deixaram de dar 403 (autorizadas), mas respondem 404/422. O vazamento de campos é coberto por response_model + testes + auditoria do OpenAPI.
- A página pós-login (/web/agenda) não foi visitada — o login por formulário não foi roteirizado no ZAP. Cookies/CSRF foram avaliados na página de login; o XSS da agenda está provado por teste e captura de tela.
- O alvo rodou em HTTP: o HSTS está presente, mas só tem efeito sob HTTPS (TLS termina no proxy).
- Executado numa máquina de desenvolvimento, não em staging; repetir no pipeline contra o ambiente de destino é a condição 1 do deploy (item 13.6).

13.5 Rastreabilidade ponta a ponta

Ameaça (Ex. 4) → controle (código) → teste (Ex. 12/13) → categoria OWASP → regra ZAP que a verifica (resultado do scan real do item 13.4). A matriz completa e executada está em docs/matriz_ameaca_teste.md (Anexo B) e a correlação do ZAP em docs/ex13_zap_correlacao.md (Anexo D); exemplos:

Ameaça	Controle	Teste	OWASP	Regra ZAP
T-I2 BOLA de leitura (MU-01)	auth/ownership.py::get_appointment_or_404	test_ex06::test_profissional_nao_le_consulta_de_outro_profissional, test_ex09::test_bola_em_todos_os_endpoints_por_id, ataque A2	A01 / API1	— o DAST passivo não enxerga autorização; só viu 403/404 (100000), aceitos. Coberta por teste
T-T3 XSS armazenado	whitelist + autoescape + CSP	test_ex02::test_pagina_agenda_escapa_conteudo_malicioso_persistido, ataque A7	A03	10038 (CSP): PASS
T-X2 CORS/headers	core/security_headers.py, CORSMiddleware	test_ex10::test_cabecalhos_obrigatorios_em_todas_as_respostas, ...::test_coep_presente_..., ataque A10	A05	10020, 10021, 10035, 10098: PASS; 90004: achado → corrigido → PASS
T-S4/T-E3 CSRF e sessão	core/csrf.py, cookie HttpOnly/SameSite/Secure	test_ex02::test_cookie_de_sessao_httponly_samesite_strict, test_ex12::test_logout_exige_csrf	A01 / A05	10202, 10010, 10011, 10054: PASS na página de login
T-S2 força bruta	rate limit 3 baldes + MFA	test_ex10::test_login_e_limitado_..., ataque A8	A07	— (ativo; fora do passivo)

13.6 Risco residual e decisão de deploy

Registro completo em docs/ex13_risco_residual.md (Anexo E). Decisão: autorizar o deploy de forma **CONDICIONAL (GO com 7 condições)**. Nenhum risco residual identificado, isoladamente, exige bloquear a liberação — desde que as condições sejam cumpridas; se qualquer uma falhar, a decisão passa a NO-GO.

Risco residual	Por que não foi eliminado	Aceitável?
R-01 CPF e prontuário em texto claro no banco	criptografia de coluna exige gestão de chaves (KMS) fora do escopo	Sim, com condição: volume/banco e backups criptografados em repouso e acesso restrito e auditado. Sem criptografia em repouso na plataforma → bloqueia.
R-02 Rate limit em memória do processo	contador por processo	Sim com 1 réplica; com N réplicas o limite efetivo é Nx → mover para o gateway/Redis antes de escalar

Risco residual	Por que não foi eliminado	Aceitável?
R-03 JWT HS256 sem revogação individual nem rotação	simplicidade/escopo	Sim: TTL 30 min + checagem no banco (desativar conta corta o acesso); roadmap RS256/EdDSA com <code>kid</code>
R-06 Auditoria sem imutabilidade	log em stdout	Sim desde que enviada a um SIEM/armazenamento WORM no go-live
R-07 M2M com segredo compartilhado (sem mTLS)	escopo	Sim: dano limitado a horários livres, TTL 10 min
R-08 Sem limite de corpo no app	cabe ao proxy	Sim com <code>client_max_body_size</code> no proxy
R-11 ZAP rodou só em máquina de desenvolvimento, passivo; IAST não executado	ambiente	ZAP: aceitável desde que repetido no pipeline contra o ambiente de destino (condição 1); IAST: lacuna aceita

Condições do GO: (1) o ZAP, já executado localmente sem alertas bloqueantes (item 13.4), **repetido no job** `dast-passive` **contra o ambiente de destino**, também sem alertas bloqueantes; (2) TLS no proxy com HSTS e `COOKIE_SECURE=true`, `ENABLE_DOCS=false` (`APP_ENV=production` exige); (3) segredos em *secret manager*, rotacionados; (4) banco e *backups* criptografados em repouso; (5) auditoria enviada a SIEM; (6) 1 réplica **ou** rate limit no gateway/Redis; (7) `client_max_body_size` no proxy e PostgreSQL com migrações no lugar de SQLite se houver mais de um nó.

Anexo A — Priorização CVSS + impacto de negócio

Scores calculados por `scripts/cvss.py` (fórmulas da especificação FIRST CVSS v3.1); os vetores são avaliação do autor para a versão **antes** das correções — não são scores oficiais do NVD.

Critério de impacto de negócio (dados de saúde — LGPD art. 5º II e art. 11)

Nível	Critério
4 — Crítico	Exposição, alteração ou controle de dados de saúde de vários pacientes/terceiros ou de contas administrativas; exige notificação à ANPD/titulares (LGPD art. 48) e pode paralisar a clínica.
3 — Alto	Compromete sessão/conta de equipe clínica ou a disponibilidade do agendamento; dado de saúde só alcançável com passos adicionais.
2 — Médio	Facilita ataques ou reprova em auditoria, sem acesso direto a dado de saúde.
1 — Baixo	Informação técnica sem efeito direto.

Regra de prioridade

- P0** — CVSS $\geq 9,0$, **ou** CVSS $\geq 7,0$ com impacto $\geq 3 \rightarrow$ corrige antes de qualquer release;
- P1** — CVSS $\geq 7,0$, **ou** impacto = 4 (mesmo com CVSS < 7 : ex. BOLA de leitura, CVSS 6,5 mas dado de saúde de terceiro);
- P2** — CVSS 4,0–6,9 · **P3** — CVSS $< 4,0$.

O CVSS mede gravidade técnica; **não enxerga** o que significa vazar um prontuário. Por isso o impacto de negócio entra na regra, e por isso as falhas de autorização têm testes que bloqueiam o pipeline *independentemente* de score.

Tabela (ordenada por impacto de negócio e score)

Prio	ID	Vulnerabilidade	OWASP	Vetor CVSS 3.1	Score	Severidade	Impacto de negócio	Correção	Ex.
P0	F-01	Cadastro público aceita <code>role=admin</code> (mass assignment)	A01:2021 / API3:2023	AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:H/A:H	9.8	Crítica	Crítico — Qualquer pessoa vira administrador e lê/altera dados de todos os pacientes.	Signup só para admin; <code>extra='forbid'</code> ; papel validado por Enum	Ex. 9
P0	F-02	Segredo JWT fixo no código/repositório (forja de token)	A02:2021 / A07:2021	AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:H/A:N	9.1	Crítica	Crítico — Token de admin forjado sem credencial; acesso total a dados de saúde.	Segredo via <code>BaseSettings/.env</code> (≥ 32 chars); <code>iss/aud/exp/jti</code> obrigatórios; HS256 fixo	Ex. 6/11
P0	F-03	SQL injection na busca de pacientes (query por f-string)	A03:2021	AV:N/AC:L/PR:L/UI:N/S:U/C:H/I:H/A:H	8.8	Alta	Crítico — Dump/alteração de toda a base de pacientes (CPF, telefone), inclusive de outros profissionais.	<code>select().where()</code> parametrizado + whitelist do termo + LIKE com <code>autoescape</code>	Ex. 9/11
P0	F-04	BOLA de escrita: PUT/DELETE de consulta de outro profissional	A01:2021 / API1:2023	AV:N/AC:L/PR:L/UI:N/S:U/C:H/I:H/A:N	8.1	Alta	Crítico — Adulteração do prontuário e cancelamento de consultas alheias (integridade clínica).	Filtro de ownership dentro da query (<code>auth/ownership.py</code>); 404 uniforme	Ex. 9
P1	F-05	BOLA de leitura: prontuário de outro profissional por ID na URL	A01:2021 / API1:2023	AV:N/AC:L/PR:L/UI:N/S:U/C:H/I:N/A:N	6.5	Média	Crítico — Vazamento de dado de saúde de terceiro (LGPD art. 11, notificação à ANPD) — CVSS 'médio', impacto crítico.	Mesmo filtro de ownership; auditoria de tentativa (<code>object_access_denied</code>)	Ex. 9
P1	F-06	BOLA no mesmo padrão em GET <code>/patient/{id}</code> (+ CPF completo)	A01:2021 / API1:2023	AV:N/AC:L/PR:L/UI:N/S:U/C:H/I:N/A:N	6.5	Média	Crítico — Identificação de pacientes de outro profissional; CPF é identificador forte.	<code>scope_patients()</code> + máscara de CPF no response model	Ex. 9
P0	F-08	Dependências com CVEs conhecidas (FastAPI/Starlette, python-multipart, python-jose)	A06:2021	AV:N/AC:L/PR:N/UI:N/S:U/C:N/I:N/A:H	7.5	Alta	Alto — ReDoS em parsing de formulário público derruba o agendamento (disponibilidade).	Versões atualizadas e fixadas; PyJWT no lugar de python-jose; pip-audit no CI	Ex. 12

Prio	ID	Vulnerabilidade	OWASP	Vetor CVSS 3.1	Score	Severidade	Impacto de negócio	Correção	Ex.
P0	F-07	Força bruta no login (sem rate limit nem MFA)	A07:2021	AV:N/AC:H/PR:N/UI:N/S:U/C:H/I:H/A:N	7.4	Alta	Alto — Tomada de contas de equipe clínica; admin sem 2º fator seria comprometimento total.	5/min por conta + 20/min por IP (429); bcrypt; MFA TOTP obrigatório para admin	Ex. 6/10
P2	F-09	XSS armazenado na agenda da recepção	A03:2021	AV:N/AC:L/PR:L/UI:R/S:C/C:L/I:L/A:N	5.4	Média	Alto — Script roda na sessão de quem abre a agenda e pode agir como recepcionista (dados de pacientes).	Whitelist de entrada + autoescape do Jinja2 + CSP sem script + cookie HttpOnly	Ex. 2/9
P2	F-10	Resposta devolve campos internos de auditoria e CPF completo	API3:2023	AV:N/AC:L/PR:L/UI:N/S:U/C:L/I:N/A:N	4.3	Média	Médio — Facilita mapeamento do sistema e identificação de usuários por terceiros.	Response models Pydantic com whitelist de campos	Ex. 2
P2	F-11	CORS com curinga e ausência de cabeçalhos de segurança	A05:2021	AV:N/AC:H/PR:N/UI:R/S:U/C:L/I:L/A:N	4.2	Média	Médio — Reprovação em auditoria de pré-produção; leitura cross-origin por site malicioso.	Allowlist de origens; HSTS, X-Frame-Options, X-Content-Type-Options, CSP	Ex. 10

Anexo B — Matriz ameaça → teste (executada)

Gerada por `python scripts/gen_traceability.py`; cada teste foi executado e o resultado é real.

ID	Ameaça	Teste(s) que a cobrem	Resultado
T-S1	JWT forjado/adulterado/expirado	test_ex06_authz > test_token_expirado_e_rejeitado test_ex06_authz > test_token_com_assinatura_errada_e_rejeitado test_ex06_authz > test_token_alg_none_e_rejeitado test_ex06_authz > test_token_de_outra_audiencia_e_rejeitado test_ex06_authz > test_token_alterado_para_papel_admin_nao_eleva_privilegio	✓ ✓ ✓ ✓ ✓
T-S2	Força bruta / credential stuffing no login	test_ex10_hardening > test_login_e_limitado_a_5_tentativas_por_janela test_ex12_threat_vectors > test_limite_por_conta_vale_mesmo_trocando_de_ip test_ex12_threat_vectors > test_credential_stuffing_de_um_ip_e_contido test_ex12_threat_vectors > test_atacante_nao_consegue_travar_o_login_do_usuario_legitimo_em_outro_ip	✓ ✓ ✓ ✓ ✓
T-T1	Mass assignment de role no cadastro	test_ex09_input_validation > test_signup_nao_aceita_campos_extras_e_exige_admin test_ex06_authz > test_usuario_sem_papel_admin_nao_acessa_rota_restrita_a_admin	✓ ✓
T-R1	Falhas de login e negações sem registro	test_ex12_threat_vectors > test_falha_de_login_e_negacao_de_acesso_geram_trilha_de_auditoria	✓
T-I1	Enumeração de usuários (mensagem/tempo)	test_ex06_authz > test_login_invalido_usa_mensagem_generica test_ex13_unitmocks > TestCriptografiaComMocks > test_login_de_usuario_inexistente_gasta_o_mesmo_custo_de_hash	✓ ✓
T-D1	Inundação de tentativas de login	test_ex10_hardening > test_token_endpoint_tambem_tem_limite_estrito test_ex13_unitmocks > test_rate_limiter_janela_deslizante_libera_apos_a_janela	✓ ✓
T-E1	Token de desafio MFA como sessão / replay do TOTP	test_ex06_authz > test_admin_exige_mfa_e_nao_recebe_access_token_so_com_senha test_ex06_authz > test_mfa_codigo_correto_emite_token_e_replay_e_negado	✓ ✓
T-S3	Falsificar o dono da consulta (professional_id no corpo)	test_ex09_input_validation > test_criar_consulta_rejeita_campos_nao_declarados test_ex01_appointments > test_profissional_cria_e_lista_consulta_do proprio_paciente	✓ ✓
T-T2	BOLA em alteração/cancelamento e mass assignment de status/IDs	test_ex09_input_validation > test_bola_em_todos_os_endpoints_por_id test_ex09_input_validation > test_atualizar_consulta_nao_aceita_trocar_dono_ou_paciente	✓ ✓
T-R2	Alteração/cancelamento sem rastro	test_ex12_threat_vectors > test_alteracoes_de_consulta_geram_trilha_de_auditoria	✓
T-I2	BOLA de leitura e campos internos de auditoria vazando	test_ex06_authz > test_profissional_nao_le_consulta_de_outro_profissional test_ex02_response_models_xss > test_nenhum_endpoint_de_consulta_vaza_campos_internos	✓ ✓
T-D2	Listagens sem limite (consumo de recursos)	test_ex12_threat_vectors > test_paginacao_tem_teto_e_rejeita_valores_abusivos	✓
T-E2	Perfil agindo fora do papel (recepção cria consulta)	test_ex06_authz > test_recepcionista_nao_cria_consulta test_ex06_authz > test_profissional_nao_agenda_para_paciente_de_outro	✓ ✓
T-S4	CSRF em login/logout da página	test_ex02_response_models_xss > test_login_web_sem_csrf_e_negado test_ex12_threat_vectors > test_logout_exige_csrf	✓ ✓
T-T3	XSS armazenado na agenda	test_ex02_response_models_xss > test_pagina_agenda_escapa_conteudo_malicioso_persistido test_ex09_input_validation > test_texto_livre_fora_da_whitelist_e_rejeitado	✓ ✓
T-I3	Dados de saúde em excesso na página / cache	test_ex02_response_models_xss > test_pagina_agenda_nao_expoe_dados_confidenciais test_ex10_hardening > test_cabecalhos_obrigatorios_em_todas_as_respostas	✓ ✓
T-E3	Roubo de sessão via cookie	test_ex02_response_models_xss > test_cookie_de_sessao_httponly_samesite_strict	✓
T-S5	Personificação do cliente M2M (segredo errado)	test_ex07_m2m_scopes > test_token_endpoint_recusa_pedidos_invalidos	✓
T-T4	Ampliar escopo no pedido de token	test_ex07_m2m_scopes > test_token_endpoint_recusa_pedidos_invalidos test_ex07_m2m_scopes > test_token_m2m_com_outro_escopo_nao_acessa_availability	✓ ✓
T-I4	Laboratório lendo dados de pacientes	test_ex07_m2m_scopes > test_laboratorio_consulta_horarios_livres_sem_dados_de_paciente test_ex07_m2m_scopes > test_token_do_laboratorio_nao_acessa_rotas_de_usuarios	✓ ✓

ID	Ameaça	Teste(s) que a cobrem	Resultado
T-E4	Token M2M em rotas de usuário e vice-versa	test_ex07_m2m_scopes > test_token_do_laboratorio_nao_acessa_rotas_de_usuarios test_ex07_m2m_scopes > test_token_de_profissional_nao_acessa_rota_do_laboratorio	✓ ✓
T-T5	SQL injection	test_ex09_input_validation > test_busca_com_payload_sql_e_barrada_pela_whitelist test_ex09_input_validation > test_payload_que_passa_na_whitelist_e_neutralizado_pela_parametrizacao test_ex09_input_validation > test_nenhum_codigo_monta_sql_por_concatenacao test_ex11_persistence > test_consulta_usa_parametros_ligados	✓ ✓ ✓ ✓
T-I5	Vazamento por credenciais no código / dump do banco	test_ex11_persistence > test_codigo_nao_contem_credenciais_embutidas test_ex11_persistence > test_settings_exigem_credenciais_do_ambiente test_ex02_response_models_xss > test_usuario_nunca_devolve_hash_nem_semente_mfa	✓ ✓ ✓
T-X1	Endpoint novo esquecido sem autenticação (deny-by-default)	test_ex12_threat_vectors > test_deny_by_default_em_todas_as_rotas	✓
T-X2	CORS permissivo / ausência de headers de segurança	test_ex10_hardening > test_origem_nao_listada_nao_recebe_cors test_ex10_hardening > test_configuracao_recusa_cors_curinga_ou_malformado test_ex10_hardening > test_cabecalhos_obrigatorios_em_todas_as_respostas	✓ ✓ ✓

53/53 verificações de teste passaram (25 ameaças mapeadas).

Anexo C — Auditoria da especificação OpenAPI

Spec: API de Agendamento Clínico v1.0.0 · OpenAPI 3.1.0 · 20 operações em 18 caminhos. Gerado por python scripts/audit_openapi.py --write.

Check	Resultado	Observação
O-01	✓ PASS	15 de 20 operações exigem autenticação; públicas = ['GET /', 'GET /health', 'POST /oauth/token', 'POST /user/mfa/verify', 'POST /user/signin']
O-02	✓ PASS	todos os corpos JSON declaram additionalProperties: false (extra='forbid')
O-03	✓ PASS	toda string de entrada JSON tem pattern, maxLength, enum ou format
O-04	✓ PASS	IDs têm mínimo (>0) e limit tem máximo (paginação limitada)
O-05	✓ PASS	todas as respostas 2xx têm schema definido (response_model)
O-06	✓ PASS	nenhum campo interno (hash, semente MFA, auditoria) aparece em schema de resposta
O-07	✓ PASS	operações protegidas documentam 401 e 429 (e 403/404 onde aplicável)
O-08	✓ PASS	esquemas: ['LabClientCredentials', 'OAuth2PasswordBearer']; fluxos: ['clientCredentials', 'password']; escopos M2M: ['availability:read']; credencial em query string: False
O-09	📘 INFO	spec não declara servers ; em produção publicar apenas https:// (TLS e HSTS no proxy)
O-10	✓ PASS	campos de formulário limitados
O-11	✓ PASS	PUT/PATCH/DELETE existem apenas sobre recurso identificado; sem TRACE
O-12	✓ PASS	operationIds únicos e todas as operações com tag

Anexo D — Correlação com o scan do ZAP

Regra ZAP	Alerta	Risco	CWE	Ocorr.	Encontrado em	OWASP Top 10	Controle no código	Teste
10049	Non-Storable Content	Informativo	524	17	api_helena_report, api_recepcao_report, baseline_report, baseline_web_report	A05:2021 Security Misconfiguration	Cache-Control: no-store em respostas com dado de saúde — core/security_headers.py	test_ex10_hardening::test_cabecalhos_obrigatorios
10111	Authentication Request Identified	Informativo	-1	3	api_helena_report, api_recepcao_report, baseline_web_report	A07:2021 Identification and Authentication Failures	login protegido por rate limit, bcrypt e MFA — routes/users.py	test_ex10_hardening::test_login_e_limitado_a_5_ter
10112	Session Management Response Identified	Informativo	-1	1	baseline_web_report	A07:2021 Identification and Authentication Failures	cookie csrf_token HttpOnly + SameSite=Strict + Secure — core/csrf.py	test_ex02_response_models_xss::test_login_web_sem
10049s	Storable and Cacheable Content	Informativo	524	1	baseline_web_report	A05:2021 Security Misconfiguration	/static/ (CSS público, sem dado de saúde) é cacheável de propósito — main.py / core/security_headers.py	test_ex10_hardening::test_cabecalhos_obrigatorios

Regra ZAP	Alerta	Risco	CWE	Ocorr.	Encontrado em	OWASP Top 10	Controle no código	Teste
100000	A Client Error response code was returned by the server	Informativo	388	30	api_helena_report, api_recepcao_report	A01:2021 Broken Access Control	RBAC, ownership, validação extra='forbid' — auth/rbac.py, auth/ownership.py	test_ex06_authz / test_ex09_input_validation

Controles comprovados pelo silêncio do ZAP

Regras que **rodaram** (PASS no log do scan) e **não** geraram alerta — a ausência é a evidência de que o controle funciona nas URLs varridas.

Regra ZAP	OWASP	Controle verificado
10020	A05:2021 Security Misconfiguration	X-Frame-Options: DENY + CSP frame-ancestors — core/security_headers.py
10021	A05:2021 Security Misconfiguration	X-Content-Type-Options: nosniff — core/security_headers.py
10035	A05:2021 Security Misconfiguration	Strict-Transport-Security — core/security_headers.py
10038	A05:2021 Security Misconfiguration	Content-Security-Policy restritiva — core/security_headers.py
10098	A05:2021 Security Misconfiguration	CORS com allowlist explícita — main.py / core/config.py
10010	A05:2021 Security Misconfiguration	Cookie de sessão HttpOnly — routes/web.py
10011	A02:2021 Cryptographic Failures	Cookie Secure (obrigatório em produção) — core/config.py
10054	A01:2021 Broken Access Control	Cookie SameSite=Strict — routes/web.py
10202	A01:2021 Broken Access Control	Token CSRF (double-submit) — core/csrf.py
10036	A05:2021 Security Misconfiguration	uvicorn --no-server-header
10037	A05:2021 Security Misconfiguration	Sem X-Powered-By
90022	A05:2021 Security Misconfiguration	Handler de erros sem eco do input / sem stack trace — main.py
10015	A05:2021 Security Misconfiguration	Cache-Control: no-store em respostas com dados de saúde
10063	A05:2021 Security Misconfiguration	Permissions-Policy — core/security_headers.py
90004	A05:2021 Security Misconfiguration	COOP + COEP + CORP (isolamento de site) — core/security_headers.py

Anexo E — Registro de risco residual

Riscos que **não foram totalmente eliminados** ao final do Assessment, com probabilidade (P) e impacto (I) de 1 (baixo) a 4 (crítico) e a decisão de aceitação.

ID	Risco residual	P	I	Compensação existente	Decisão	Condição / ação
R-01	CPF, telefone e prontuário em texto claro no banco/backup (sem criptografia de coluna)	1	4	API devolve CPF mascarado; acesso ao banco só pela aplicação; hash/segedros protegidos	Aceito com condição	Banco e <i>backups</i> criptografados em repouso + acesso restrito e auditado. Sem isso → BLOQUEIA o deploy. Próximo ciclo: criptografia de coluna com chave em KMS.
R-02	Rate limit em memória do processo	2	3	3 baldes; bcrypt; MFA admin	Aceito (1 réplica)	Com N réplicas o limite efetivo é Nx: mover contadores para Redis/gateway antes de escalar.
R-03	JWT HS256 com segredo compartilhado; sem revogação individual nem rotação (kid)	2	3	TTL 30 min; get_current_user consulta o banco (desativar conta corta acesso na hora)	Aceito	Roadmap: RS256/EdDSA + JWKS + rotação; <i>denylist</i> por <i>jti</i> se exigirem logout imediato.
R-04	Bloqueio por botnet : 15 tentativas/min em várias origens travam o login de uma conta	2	2	Chave composta conta+IP impede bloqueio por um atacante (testado)	Aceito	Monitorar <i>rate_limit</i> na auditoria; CAPTCHA/step-up se ocorrer.
R-05	MFA simulado : sem QR Code, <i>backup codes</i> , recuperação; TOTP é phishável	2	3	Só <i>admin</i> ; anti-replay; segredo derivado fora do banco	Aceito	Evoluir para WebAuthn/IdP (Keycloak, aula 15).
R-06	Trilha de auditoria em stdout , não imutável	2	3	JSON estruturado; sem segredos/PHI	Aceito com condição	Enviar a SIEM/armazenamento WORM no go-live.
R-07	Cliente M2M autenticado por segredo compartilhado (sem mTLS/ <i>private_key_jwt</i> , sem allowlist de IP)	2	2	Escopo mínimo (só horários), TTL 10 min, segredo só como hash, limite e auditoria	Aceito	Exigir mTLS/IP allowlist no contrato de renovação.
R-08	Sem limite de tamanho de corpo no app	2	2	Validação por campo (<i>max_length</i>); paginação	Aceito com condição	<i>client_max_body_size</i> (ex.: 256 KB) no proxy.
R-09	SQLite em arquivo (concorrência, HA, <i>backup</i>)	2	3	Camada configurável via DATABASE_URL	Aceito (piloto, 1 nó)	PostgreSQL + migrações Alembic para produção multi-nó.
R-10	Whitelist rejeita caracteres legítimos (< , & , dígitos em nomes)	3	1	Mensagem de erro clara	Aceito	Revisar com a equipe clínica; ajustar regex.
R-11	ZAP rodou só em máquina de desenvolvimento (passivo, 4 scans; 0 bloqueantes) e IAST não foi executado	2	3	Pipeline com <i>dast-passive</i> ; 197 testes cobrem a lógica que o DAST passivo não vê	Aceito com condição	Repetir o <i>dast-passive</i> contra o ambiente de destino (staging) antes do go-live; zero alertas bloqueantes .
R-12	IAST não implantado	2	2	Testes de autorização + ZAP autenticado	Aceito	Avaliar agente IAST em QA.
R-13	CVEs futuras em dependências	3	3	pip-audit/Trivy no PR e semanal	Aceito	Responder a alertas do gate em até 7 dias (HIGH/CRITICAL).

ID	Risco residual	P	I	Compensação existente	Decisão	Condição / ação
R-14	Datas de consulta são horário local do servidor (sem fuso: <code>NaiveDatetime</code>)	2	1	Validação de futuro e de bloco de 30 min; uma clínica, um fuso	Aceito	Fixar <code>TZ</code> do processo/contêiner; guardar fuso por unidade se houver clínicas em fusos diferentes.

Decisão

GO condicional. Nenhum risco acima, isoladamente, justifica bloquear a liberação — todos têm compensação ativa e plano de evolução — **desde que as 7 condições abaixo estejam satisfeitas.** A falha de qualquer uma muda a decisão para **NO-GO**.

- 1. ZAP (já executado localmente: 0 bloqueantes) **repetido no job** `dast-passive` **contra o ambiente de destino**, sem alertas bloqueantes (R-11).
- 2. TLS no proxy com HSTS; `APP_ENV=production` (exige `COOKIE_SECURE=true`, `ENABLE_DOCS=false`, `bcrypt ≥ 12`).
- 3. Segredos (`JWT_SECRET_KEY`, `MFA_MASTER_KEY`, hash do M2M) em *secret manager*, rotacionados e fora do repositório.
- 4. Banco e *backups* criptografados em repouso (R-01).
- 5. Auditoria enviada a SIEM/WORM (R-06).
- 6. Uma réplica **ou** rate limit no gateway/Redis (R-02).
- 7. `client_max_body_size` no proxy; PostgreSQL e migrações se houver mais de um nó (R-08, R-09).

O risco que mais pesa é o R-01, porque o dado é de saúde: sua aceitação é **a única que depende de uma condição de plataforma** (criptografia em repouso), e é a que eu transformaria em item bloqueante caso a plataforma de destino não a ofereça.

Anexo F — Índice de evidências

Tudo aqui é saída **real**, regenerável com `bash scripts/collect_evidence.sh` (o ZAP, que exige Docker, tem `scripts/run_zap_passive.sh`).

Ex.	Pasta	O que comprova
1	<code>ex01_fundacao/</code>	<code>pytest_ex01.txt</code> (teste de consulta), <code>pip_freeze_venv.txt</code> (ambiente isolado), <code>swagger_docs.png</code> (rotas públicas x protegidas)
2	<code>ex02_templates_xss/</code>	<code>agenda_normal.png</code> , <code>agenda_xss_escapado.png</code> (payload persistido exibido como texto), <code>login.png</code> , HTMLs, <code>pytest_ex02.txt</code>
3 e 5	<code>ex03_ex05_diagramas/</code>	<code>dfd_nivel1.png/.svg</code> (DFD + trust boundaries + fluxos sensíveis), <code>arquitetura_particoes.png/.svg</code> (partições e 3 eixos)
4	<code>../docs/ex04_threat_model.md</code>	misuse cases, STRIDE (5 componentes), DREAD, árvore de ataque, superfícies
6	<code>ex06_autenticacao_autorizacao/</code>	<code>pytest_ex06.txt</code> (inclui o teste "não-admin barrado em rota de admin")
7	<code>ex07_m2m_escopos/</code>	<code>fluxo_client_credentials_curl.txt</code> (token, claims, 403, <code>invalid_scope</code>), <code>pytest_ex07.txt</code>
8 e 9	<code>ex08_ex09_antes_depois/</code>	<code>attack.py</code> (payloads), <code>resultado_antes.txt/json</code> (10/10 explorados), <code>resultado_depois.txt/json</code> (0/10), <code>antes_depois.png</code> , <code>vulnerable_baseline/</code> (app propositalmente vulnerável — não executar fora de localhost), <code>pytest_ex09.txt</code>
10	<code>ex10_hardening/</code>	<code>headers_cors_curl.txt</code> , <code>rate_limit_login_curl.txt</code> (5x401 → 429), <code>pytest_ex10.txt</code>
11	<code>ex11_persistencia/</code>	<code>pytest_ex11.txt</code> ; ver também <code>../.env.example</code>
12	<code>ex12_pipeline/</code>	<code>security-pipeline.yml</code> , <code>bandit_antes/depois.txt</code> , <code>pip_audit_antes/depois.txt</code> (+ <code>.json</code>), <code>starter_kit_requirements.txt</code> , <code>gate_local_saida.txt</code> , <code>pytest_ex12.txt</code> ; tabela CVSS em <code>../docs/cvss_priorizacao.md</code>
1	<code>ex01_fundacao/uvicorn_e_rotas.txt</code>	log do uvicorn subindo + <code>curl</code> das rotas (rubrica 1.1)
2	<code>ex02_templates_xss/sem_vs_com_response_model.txt</code>	mesmo registro sem e com <code>response_model</code> (rubrica 1.3)
13	<code>ex13_capstone/</code>	<code>pytest_suite_completa_com_cobertura.txt</code> (197 passed, 96 %), <code>pytest_ex13_unit_mock_openapi.txt</code> , <code>e2e_demo.txt</code> , <code>openapi.json</code> , <code>zap/</code> (4 scans reais do ZAP + antes/depois da correção do achado 90004); auditoria em <code>../docs/ex13_openapi_audit.md</code> ; risco residual em <code>../docs/ex13_risco_residual.md</code>

`evidence/.local/` (credenciais de demonstração, segredo do laboratório, logs, bancos locais) **não** entra no ZIP.

Anexo G — Mapa da rubrica de competências → evidência

Cada item da rubrica de competências, com o ponto exato do relatório, o código, o teste e a evidência. Status:  demonstrado e executado ·  demonstrado, com uma etapa externa ao ambiente de desenvolvimento ainda a registrar.

Competência 1 — APIs REST com FastAPI aplicando fundamentos de segurança

#	Item da rubrica	Onde (relatório)	Código / teste	Evidência
1.1	Ambiente virtual isolado, uvicorn evidenciando a resposta das rotas , módulos <code>routes/models/database</code>	Ex. 1	<code>main.py</code> , <code>routes/</code> , <code>models/</code> , <code>database/</code> ; <code>tests/test_ex01_appointments.py</code>	<code>evidence/ex01_fundacao/uvicorn_curl</code> das rotas, <code>pip_freeze_venv</code>
1.2	<code>response_model</code> controlando exatamente os campos expostos	Ex. 2	<code>models/*.py</code> (<code>AppointmentPublic</code> , <code>AgendaItem</code> , <code>PatientPublic</code> ...); <code>test_ex02::test_nenhum_endpoint_de_consulta_vaza_campos_internos</code>	<code>evidence/ex02_templates_xss/</code>

#	Item da rubrica	Onde (relatório)	Código / teste	Evidência
1.3	Justificar o risco de exposição quando response_model não é definido	Ex. 2 (subseção "O risco de não declarar response_model")	ataque A9; auditoria OpenAPI O-06	sem_vs_com_response_model.tx resultado_antes.txt (A9)
1.4	Jinja2 com herança e auto-escape prevenindo XSS	Ex. 2	templates/base.html → login/agenda; routes/web.py; test_ex02::test_pagina_agenda_escapa_conteudo_malicioso_persistido	agenda_xss_escapado.png
1.5	Triade CIA + frameworks (OWASP, NIST SSDF, MITRE) → controles	Ex. 3 (3.1 e 3.2)	tabelas com arquivo e teste por linha	—
1.6	DFD com trust boundaries e fluxos sensíveis	Ex. 3 (3.3)	scripts/make_diagrams.py	ex03_ex05_diagramas/dfd_nive

Competência 2 — Threat modeling STRIDE e autenticação JWT

#	Item da rubrica	Onde	Código / teste	Evidência	
2.1	Misuse cases de vetores reais da própria aplicação	Ex. 4 · docs/ex04_threat_model.md §3	MU-01...MU-09, cada um com ataque executável (A1–A10)	resultado_antes.txt	✓
2.2	STRIDE + threat model completo (ativos, superfícies, mitigações rastreáveis)	Ex. 4 · §1, §2, §4	tests/threat_matrix.py (25 ameaças → 53 verificações)	docs/matriz_ameaca_teste.md	✓
2.3	Fronteiras de segurança e vetores nos 3 eixos de API	Ex. 5 · docs/ex04_threat_model.md §7	diagrama de partições	ex03_ex05_diagramas/arquitetura_particoes.png	✓
2.4	OAuth2PasswordBearer + bcrypt + rotas com ownership	Ex. 6	auth/authenticate.py, auth/hash_password.py, auth/ownership.py; test_ex06	pytest_ex06.txt	✓
2.5	JWT com expiração, MFA simulado, justificar RBAC x ABAC x por recurso	Ex. 6 (tabela comparativa)	auth/jwt_handler.py, auth/mfa.py; test_ex06 (expirado, replay, MFA web)	pytest_ex06.txt	✓
2.6	Fluxo OAuth 2.0 adequado + escopos/claims humano x M2M	Ex. 7	routes/oauth.py, auth/rbac.py::require_scopes; test_ex07	fluxo_client_credentials_curl.txt	✓

Competência 3 — Vulnerabilidades OWASP Top 10 e controles defensivos

#	Item da rubrica	Onde	Código / teste	Evidência	
3.1	Identificar padrões vulneráveis lendo código — ≥ 3 categorias	Ex. 8 (9 achados, 6 categorias, com arquivo:linha)	evidence/.../vulnerable_baseline/app.py; código do Starter Kit	attack.py + resultado_antes.txt	✓
3.2	Identificar e corrigir BOLA com ownership centralizado	Ex. 8 e 9	auth/ownership.py (um módulo, usado por todas as rotas por ID); test_ex09::test_bola_em_todos_os_endpoints_por_id	antes_depois.png (A2–A4)	✓
3.3	Whitelist + regex + extra='forbid'	Ex. 9	models/base.py (StrictModel, padrões); test_ex09	resultado_depois.txt (A1, A5)	✓
3.4	Corrigir XSS stored com output encoding (auto-escape)	Ex. 2 e 9	routes/web.py; test_ex02 (grava <script> direto no banco)	agenda_xss_escapado.png, A7	✓
3.5	CORS com allowlist, HSTS/X-Frame-Options/X-Content-Type-Options, rate limit diferenciado	Ex. 10	core/security_headers.py, core/rate_limit.py; test_ex10	headers_cors_curl.txt, rate_limit_login_curl.txt	✓
3.6	SQLModel parametrizado + BaseSettings / .env sem hardcoding	Ex. 11	database/, core/config.py, .env.example; test_ex11	pytest_ex11.txt	✓

Competência 4 — Pipeline DevSecOps, security gates, ZAP e testes

#	Item da rubrica	Onde	Código / teste	Evidência	
4.1	Fase do SDLC por tipo de ferramenta (estática, dinâmica, dependências, interativa), com justificativa amarrada ao pipeline	Ex. 12 (12.1) · docs/ex12_pipeline_devsecops.md §2	.github/workflows/security-pipeline.yml	bandit_antes.txt	✓
4.2	Priorizar com CVSS e impacto de negócio	Ex. 12 (12.3)	scripts/cvss.py (validado por teste)	docs/cvss_priorizacao.md	✓

#	Item da rubrica	Onde	Código / teste	Evidência	
4.3	Security gate em GitHub Actions com critério definido e justificado pelo aluno, impedindo merge	Ex. 12 (12.2 e 12.4)	job security-gate + gate-regression; scripts/ci_gate.py; branch protection descrita	security-pipeline.yml, gate_local_saida.txt	🔴 workflow validado e comandos executados localmente; falta o print do check verde após subir o repositório
4.4	Estratégia de testes rastreado ao threat model, expandindo o teste do Ex. 6	Ex. 12 (12.5)	tests/test_ex12_threat_vectors.py, tests/threat_matrix.py	docs/matriz_ameaca_teste.md	✅
4.5	Executar e interpretar scan passivo do ZAP contra a própria aplicação, correlacionando findings a OWASP e à correção, com risco residual e decisão de deploy	Ex. 13 (13.4–13.6)	scripts/run_zap_passive.sh, scripts/zap_report.py, core/security_headers.py (correção do 90004)	evidence/ex13_capstone/zap/ (4 scans reais, antes e depois da correção), docs/ex13_zap_correlacao.md, docs/ex13_risco_residual.md	✅
4.6	Testes unitários com mocking (teste inicial do Ex. 1 + pontos de entrada e autorização) e auditoria do OpenAPI	Ex. 13 (13.2 e 13.3)	tests/test_ex13_unit_mock.py (inclui TestEx01ComMocks), scripts/audit_openapi.py	pytest_ex13_unit_mock_openapi.txt, docs/ex13_openapi_audit.md	✅

O que a rubrica pede que depende de você

- **Vídeo** (até 5 min, YouTube não listado): demonstra 1.1, 3.x e 4.x ao vivo e é a evidência de autoria — roteiro em docs/ROTEIRO_VIDEO.md.
- **Item 4.3:** subir o repositório ao GitHub, deixar o workflow rodar, salvar o *print* do check SECURITY GATE e ligar o *branch protection* exigindo-o (é isso que "impede o merge").